

MEMORIA DE PROYECTO INTERMODULAR

DESARROLLO DE UNA PLATAFORMA SINGLE PAGE APPLICATION (SPA) PARA LA GESTIÓN DE CATÁLOGOS, MÉTRICAS DE VENTAS Y AUDITORÍA DE SEGURIDAD EN ENTORNOS INDIE

Centro Educativo: IES Francisco de Quevedo

Ciclo Formativo: 2º Curso de Desarrollo de Aplicaciones Web (DAW)

Autor: Josué Rodríguez Díaz

Enlace al repositorio del proyecto: <https://github.com/JosueRodriguezDiaz/Proyecto-Intermodular-Tienda-Online>

INDICE

MEMORIA DE PROYECTO INTERMODULAR.....	1
DESARROLLO DE UNA PLATAFORMA SINGLE PAGE APPLICATION (SPA) PARA LA GESTIÓN DE CATÁLOGOS, MÉTRICAS DE VENTAS Y AUDITORÍA DE SEGURIDAD EN ENTORNOS INDIE.....	1
CAPÍTULO 1: INTRODUCCIÓN.....	2
CAPÍTULO 2: OBJETIVOS DEL PROYECTO.....	2
2.1 Objetivo general.....	2
2.2 Objetivos específicos.....	2
CAPÍTULO 3: ESTADO DEL ARTE.....	3
3.1 Cómo se resuelve este problema en empresas reales.....	3
3.2 Herramientas y tecnologías habituales.....	3
CAPÍTULO 4: PLANTEAMIENTO Y ARQUITECTURA DEL PROYECTO.....	3
4.1 Tecnologías Utilizadas e Integración de Módulos.....	3
CAPÍTULO 5: DESARROLLO DEL PROYECTO (PARTE EXPERIMENTAL).....	4
5.1 El Entorno de Usuario Comprador (USER).....	4
5.2 El Entorno del Desarrollador (DEV).....	4
5.3 El Entorno del Administrador (ADMIN).....	4
CAPÍTULO 6: CONCLUSIONES.....	5
6.1 Aprendizaje técnico.....	5
6.2 Valoración global del proyecto.....	5
BIBLIOGRAFÍA.....	5
ANEXOS TÉCNICOS.....	5
Modelo Entidad-Relación Lógico (Diseño para Escalabilidad Futura).....	5
Descripción de Relaciones de Integridad:.....	6
Anexo A: Estructura de Persistencia de Datos NoSQL (juegos.json y comentarios.json).....	6
1. Estructura del Fichero juegos.json:.....	6
2. Estructura del Fichero comentarios.json:.....	6
Anexo B: Automatización y Política de Respallos de Servidor por Replicación Física.....	7
Código del Script de Backup Automatizado (backup_sistema.sh):.....	7

CAPÍTULO 1: INTRODUCCIÓN

En esta memoria expongo el diseño, desarrollo e implementación de mi proyecto final: una plataforma web avanzada orientada a la distribución de videojuegos y la automatización de flujos de interacción entre desarrolladores independientes y usuarios compradores. La aplicación ha sido concebida bajo el paradigma de desarrollo moderno **Single Page Application (SPA)**, lo que significa que toda la experiencia de navegación ocurre dentro de una única página web de carga inmediata. Con esto, consigo eliminar por completo las transiciones lentas y las recargas del navegador mediante una manipulación quirúrgica y dinámica del Árbol de Objetos del Documento (DOM) controlada por mis scripts de JavaScript en el lado del cliente.

Para resolver las necesidades típicas de un entorno de software real, he estructurado la aplicación en torno a tres perfiles de acceso concurrentes y aislados mediante lógica algorítmica: el **Administrador** (responsable del soporte informático, monitorización del estado del sistema y backups), el **Desarrollador** (provisto de un cuadro de mandos financieros y control de feedback) y el **Usuario** (con acceso a catálogo, carrito de la compra y biblioteca interactiva). La persistencia de datos y el flujo de información del ecosistema se apoyan en un motor de almacenamiento NoSQL basado en archivos planos gestionados directamente por el servidor, diseñado específicamente para garantizar un despliegue ágil, autocontenido y libre de dependencias complejas de bases de datos externas en la fase local.

CAPÍTULO 2: OBJETIVOS DEL PROYECTO

2.1 Objetivo general

Construir una aplicación web SPA nativa y unificada que automatice el ciclo de vida de la distribución de software interactivo, estructurando entornos de trabajo restringidos según niveles de privilegio, y dotando al sistema de módulos analíticos financieros y de auditoría de infraestructura.

2.2 Objetivos específicos

- **Optimización Extrema de Rendimiento (Velocidad):** Reducir los tiempos de carga inicial e intercambio de secciones a menos de 1 segundo mediante la inyección dinámica de código y ocultación condicional por CSS.
- **Seguridad y Control de Roles:** Programar una función de autenticación centralizada capaz de segregar de manera estricta las funcionalidades visibles para los roles ADMIN, DEV y USER.
- **Módulo Analítico e Inyección en Tiempo Real:** Diseñar una interfaz interactiva para el desarrollador que proyecte métricas de ventas basadas en representación gráfica e incorpore la publicación asíncrona de nuevos productos.
- **Canal de Feedback con Trazabilidad:** Habilitar un sistema de comunicación bidireccional post-compra donde los compradores envíen reseñas técnicas y el autor del videojuego pueda emitir réplicas oficiales.
- **Garantías de Cumplimiento Legal (RGPD):** Asegurar la integridad informativa en la capa cliente exigiendo la aceptación explícita de las políticas de privacidad previas al almacenamiento de credenciales o accesos.

CAPÍTULO 3: ESTADO DEL ARTE

3.1 Cómo se resuelve este problema en empresas reales

En mi fase de investigación sectorial, he analizado cómo operan los grandes marketplaces de la industria como Steam (Valve) o Epic Games Store. Estas corporaciones sustentan sus masivos volúmenes de tráfico desacoplando sus arquitecturas: emplean backends distribuidos a nivel global basados en microservicios e implementan frontends basados en el concepto de Single Page Application. De este modo, evitan saturar los servidores con peticiones de páginas completas cada vez que un usuario explora el catálogo o añade un artículo a su cesta de la compra.

3.2 Herramientas y tecnologías habituales

El estándar de desarrollo industrial combina frameworks de componentes reactivos en el lado del cliente (como React, Vue.js o Angular) con servidores de aplicaciones basados en Java (Spring Boot) o C# (.NET Core). Para el almacenamiento masivo, las bases de datos relacionales como MySQL o PostgreSQL siguen siendo la opción predominante para transacciones financieras debido a sus propiedades ACID, mientras que las bases de datos orientadas a documentos (NoSQL) agilizan catálogos muy dinámicos. En producción, las políticas de contingencia exigen scripts automatizados de copia en caliente (backups) programados periódicamente mediante tareas cron en sistemas Linux.

CAPÍTULO 4: PLANTEAMIENTO Y ARQUITECTURA DEL PROYECTO

4.1 Tecnologías Utilizadas e Integración de Módulos

Para maximizar la portabilidad de la plataforma y garantizar un despliegue libre de dependencias externas complejas, se ha implementado una arquitectura Full-Stack desacoplada pero autocontenida. Los componentes tecnológicos se dividen según su responsabilidad en el patrón cliente-servidor:

- **Front-End (Cliente SPA):** Desarrollado mediante una arquitectura nativa basada en HTML5 estandarizado, CSS3 síncrono (utilizando variables nativas `:root` para la gestión de la paleta corporativa) y JavaScript Asíncrono (Vanilla JS). El intercambio de datos con el servidor se realiza mediante la API nativa `fetch`, procesando promesas para actualizar el DOM en tiempo real sin recargas de página.
- **Back-End (Servidor API REST):** Desarrollado sobre el entorno de ejecución Node.js utilizando el framework Express. Este módulo actúa como una pasarela de servicios asíncronos que expone endpoints securizados para las operaciones del catálogo, el módulo de feedback y los flujos de autenticación.
- **Módulo de Persistencia de Datos (Almacenamiento NoSQL JSON):** Con el objetivo de garantizar una disponibilidad del 100% sin depender de gestores de bases de datos relacionales externos (como MySQL bajo entornos XAMPP locales) durante las primeras etapas de despliegue, se ha diseñado un motor de persistencia integrado. Este motor utiliza el módulo nativo de sistema de archivos de Node.js (`fs`) para realizar operaciones de lectura y escritura atómicas sobre ficheros estructurados en formato JSON.

CAPÍTULO 5: DESARROLLO DEL PROYECTO (PARTE EXPERIMENTAL)

Mi prototipo de software funcional opera de forma fluida aislando las vistas mediante una lógica de discriminación de roles ejecutada en mi función de autenticación. A continuación, detallo los tres entornos que he programado:

5.1 El Entorno de Usuario Comprador (USER)

Cuando un usuario inicia sesión con credenciales estándar, mi script oculta los paneles técnicos y renderiza el catálogo general de videojuegos de forma dinámica. He programado un módulo de transacciones simuladas que permite añadir juegos a un carrito dinámico, persistido localmente en `localStorage`. Una vez confirmada la compra, el sistema asocia los juegos adquiridos dentro de la biblioteca personal del usuario. Desde este panel, el comprador dispone de un cuadro de texto interactivo para enviar una reseña asociada al ID de un videojuego comprado, inyectando un objeto de opinión con campos indexados (`id_juego`, `mensaje`, `usuario_nombre`).

5.2 El Entorno del Desarrollador (DEV)

Diseñado como un centro operativo de monitorización para el creador de software. He incorporado una gráfica analítica de ventas construida de forma nativa mediante la manipulación de alturas de barras a través de CSS dinámico. El desarrollador tiene acceso a un formulario de inyección con validación de campos vacíos; al completarse con éxito, se realiza una petición HTTP POST que inserta de forma asíncrona un nuevo juego dentro del catálogo. Asimismo, cuenta con un lector de feedback que recopila las reseñas de los jugadores sobre sus juegos y le permite escribir y publicar una réplica oficial (`respuesta_dev`).

5.3 El Entorno del Administrador (ADMIN)

Para cerrar el ciclo de control técnico de la plataforma, he programado un panel de control maestro específico para el rol de administración. Este módulo expone en tiempo real las métricas críticas de la salud del servidor informático: monitorización activa del firewall de red, estado estable de las réplicas del clúster y un sistema de control mediante un botón interactivo que permite simular la llamada asíncrona a procesos de guardado manual del estado del sistema o copias de seguridad de los ficheros de producción de la empresa.

CAPÍTULO 6: CONCLUSIONES

6.1 Aprendizaje técnico

El desarrollo integral de este software me ha permitido consolidar de manera práctica la lógica detrás de las Single Page Applications. He aprendido a gestionar estados globales de datos en el cliente sin depender de frameworks complejos (como React o Vue), comprendiendo cómo estructurar arquitecturas web limpias, fluidas y de alto rendimiento. De igual modo, he asimilado la importancia de separar las responsabilidades del front-end y del backend, garantizando que el tratamiento de la información en el cliente ocurra de forma puramente asíncrona.

6.2 Valoración global del proyecto

Considero que el sistema resultante es plenamente funcional y cumple con creces con los criterios de velocidad (carga inicial inferior a un segundo en red local) e interacción responsiva establecidos en mis especificaciones de requisitos. En un entorno real de mercado, la arquitectura frontend planteada en mi JavaScript nativo es totalmente viable y escalable; bastaría con sustituir las colecciones de datos locales en formato JSON y el sistema de lectura/escritura física (`fs`) por llamadas asíncronas (`fetch`) dirigidas a una API REST persistente conectada directamente a un esquema de base de datos relacional robusto (como MySQL), cuyo modelo lógico ya ha sido desarrollado en esta fase de diseño.

BIBLIOGRAFÍA

1. **Documentación Oficial de MDN Web Docs.** Estándares de manipulación del DOM y diseño web adaptativo mediante CSS Grid y Flexbox.
2. **Especificación Oficial ECMAScript (ES6+).** Manual de buenas prácticas en la optimización de código JavaScript nativo y control asíncrono.
3. **Manual Técnico de Referencia de MySQL 8.0.** Modelado de datos relacionales, restricciones de integridad y administración de sentencias estructuradas (utilizado para el diseño lógico de escalabilidad futura).

ANEXOS TÉCNICOS

A efectos de auditoría y validación de infraestructura, incluyo el diseño lógico y el código de soporte desarrollado para la persistencia del sistema:

Modelo Entidad-Relación Lógico (Diseño para Escalabilidad Futura)

Para prever una transición fluida hacia un entorno de producción masivo y relacional, se ha planteado la siguiente estructura de tablas que daría soporte al mapeo de datos actual:

USUARIOS		JUEGOS		COMENTARIOS	
PK id (INT)	1 0..N	PK id (INT)	1 0..N	PK id (INT)	
username (VC)		titulo (VC)		FK id_juego (INT)	
email_hash (VB)		FK id_autor (INT)		FK id_user (INT)	
password_hash(VC)		precio (DEC)		mensaje (TEXT)	
rol (ENUM)				resp_dev (TEXT)	

Descripción de Relaciones de Integridad:

- **Usuarios a Juegos (1 a 0..N):** Un usuario (con rol DEV) puede haber publicado cero o muchos juegos, pero un juego pertenece obligatoriamente a un único desarrollador.
- **Usuarios a Comentarios (1 a 0..N):** Un usuario puede escribir cero o muchos comentarios, pero cada comentario está firmado de forma unívoca por un único usuario.
- **Juegos a Comentarios (1 a 0..N):** Un juego puede almacenar en su ficha cero o muchos comentarios comunitarios, pero un comentario solo puede existir vinculado a un juego concreto.

Anexo A: Estructura de Persistencia de Datos NoSQL (juegos.json y comentarios.json)

En sustitución provisional de un modelo relacional en la base local de pruebas, el sistema gestiona la información de forma estructurada en documentos JSON de alta velocidad de lectura. El ciclo de vida de los datos se controla mediante funciones ayudantes asíncronas que leen el almacenamiento físico al arrancar (`leerDatos`) y realizan volcados atómicos al disco ante cualquier mutación de los recursos (`guardarDatos`).

1. Estructura del Fichero juegos.json:

```
[
  {
    "id": 1,
    "titulo": "TEKKEN 8",
    "precio": 14.99,
    "imagen": "IMGS/Tekken8.jpg"
  },
  {
    "id": 2,
    "titulo": "Kingdom Hearts IV",
    "precio": 99.99,
    "imagen": "IMGS/Kingdom_Hearts_IV.jpg"
  }
]
```

2. Estructura del Fichero comentarios.json:

```
[
  {
    "id": 1715964821000,
    "id_juego": 1,
    "mensaje": "El juego tiene tirones en el menú principal.",
    "respuesta_dev": "Estamos trabajando en un parche de optimización.",
    "juego_titulo": "TEKKEN 8",
    "usuario_nombre": "Invitado Nexus"
  }
]
```

Anexo B: Automatización y Política de Respaldos de Servidor por Replicación Física

Al haber implementado un almacenamiento basado en ficheros planos en el servidor (.json), la estrategia de contingencia de seguridad perimetral aplica una política de Copia de Seguridad por Replicación de Archivo Físico comprimido, programada periódicamente.

Código del Script de Backup Automatizado (backup_sistema.sh):

```
#!/bin/bash
# Script de automatización de respaldos para base NoSQL JSON

DIR_PROYECTO="/var/www/nexus_games_backend"
DIR_RESPALDO="/var/backups/nexus_db"
FECHA_ACTUAL=$(date +%F_%H-%M-%S)

# Crear directorio de respaldo si no existe
mkdir -p "$DIR_RESPALDO"

# Ejecución del volcado y compresión tarball de los archivos JSON de datos de
# producción
tar -czf "$DIR_RESPALDO/backup_datos_$FECHA_ACTUAL.tar.gz" -C "$DIR_PROYECTO"
juegos.json comentarios.json

# Mantenimiento preventivo: eliminar copias con una antigüedad superior a 30
# días
find "$DIR_RESPALDO" -name "backup_datos_*.tar.gz" -mtime +30 -exec rm {} \;
```